

German International University
Faculty of Informatics and Computer Science
Dr. John Zaki
Amir Haytham
Hassan Abdelqader
Roba Askar
Nada Farid

Software Engineering, Spring 2026
GIU Nexus — AI-Powered Career & Talent Platform
Project

1 Overview

In this project, you will build a full-stack web application that helps university students find internships and jobs while giving recruiters a smarter, faster way to discover the right candidates. The system is called **GIU Nexus**, and what makes it stand out from a typical job board is that it integrates real AI capabilities powered by the Hugging Face Inference API. Instead of students manually typing in skill tags, the system reads their profile text and figures out their skills automatically. Instead of a recruiter guessing which category a job belongs to, the AI classifies it instantly. Instead of a student scrolling through dozens of irrelevant listings, the platform surfaces the ones that best match their background.

You will be building something that is genuinely useful, technically challenging, and directly relevant to your own lives as students about to enter the workforce.

2 Project Objectives

- Gain hands-on experience designing and building a MERN-stack application from scratch.
- Practice schema design with Mongoose to model real-world relationships between users, jobs, and applications.
- Learn Git-based collaboration workflows: branching, pull requests, code reviews, and conflict resolution.
- Understand how to integrate a third-party AI API (Hugging Face) into a Node.js backend in a secure and responsible way.
- Deliver a working product in stages through structured milestones.

3 User Roles

The platform has three distinct types of users, each with a different relationship to the data.

- a) **Job Seeker (Standard User):** A university student looking for internships or entry-level positions. They can create a profile, browse approved job listings, apply to positions, track their application history, and receive AI-generated skill suggestions based on their written bio.
- b) **Recruiter (Company Representative):** A company or individual posting opportunities. Recruiters can create, edit, and delete their own job listings. When they post a job, the AI automatically assigns it a technical category. They can also view the list of applicants for each of their postings and update application statuses.

- c) **System Admin:** Has full visibility and control. Admins approve or reject newly registered recruiter accounts before those recruiters can post any jobs. They can also remove flagged listings and view platform-wide analytics.

4 Project Features

- a) **Smart Job Listings:** Each job post is automatically classified into a technical category (e.g., Frontend, Backend, AI/ML, DevOps, Data Engineering) by the Hugging Face zero-shot classification model when the recruiter creates it. This makes browsing and filtering far more reliable than relying on recruiters to pick the right tag manually.
- b) **AI Skill Extraction:** When a student writes or updates their profile bio, the backend runs it through a Named Entity Recognition (NER) model to detect technologies, tools, and domain keywords. The extracted skills are saved to the student's profile and displayed as visual chips in the UI.
- c) **Recommended Jobs:** The platform uses sentence embeddings to compute a similarity score between a student's skill set and each available job's requirements. The homepage shows each student a personalized ranked list of the top relevant jobs.
- d) **Application Tracking:** Students apply to jobs and can monitor the status of each application. Recruiters update statuses from within their dashboard. Statuses follow the flow: *pending* → *shortlisted* or *rejected*.
- e) **Recruiter Dashboard:** Recruiters see all their active postings, the number of applicants per posting, and can drill into individual applicant profiles.
- f) **Admin Panel:** Admins review pending recruiter registrations, approve or deny them, manage job listings, and view summary statistics across the platform.
- g) **Database Integration:** All users, job posts, and application records are stored in MongoDB with proper relationships and timestamps.

5 Project Milestones

- **Task 1:** Repository setup, Git workflow practice, and Mongoose schema design.
- **Task 2:** Backend development — Express server, authentication, role-based access control, CRUD operations, and Hugging Face API integration.
- **Task 3:** Frontend development — React interface for all user flows, integration with the backend, and rendering AI results in the UI.

6 Milestone 1 Description

Task 1 focuses on two things: establishing a proper collaborative Git workflow, and designing the data models that will sit at the heart of your application for the rest of the semester. Getting these right from the beginning matters — a poorly designed schema often leads to painful refactoring weeks later.

Version Control & Collaboration

- The team leader must create a **private** GitHub repository for the project and invite the TA account (SEspring26) as a collaborator.
- Each team member must create their own **feature branch** named after their assigned work area (e.g., `feat/user-schema`, `feat/job-schema`).

- Every member must push **at least one meaningful commit** — not just an empty file or a copy-paste of someone else's work.
- Before merging anything to **main**, you must open a **Pull Request**, request a review from at least one teammate, and resolve any comments or conflicts collaboratively.

The repository must also contain a `README.md` file that briefly describes the project and lists the team members, and a proper `.gitignore` that ignores `node_modules/`, `.env` files, and any IDE-specific folders.

Database Schema Design

You will implement the following three Mongoose schemas. Each one is described below. Think carefully about the field types, which fields are required, and which references link one collection to another.

1. User Schema

Everyone on the platform — students, recruiters, and admins alike — lives in the same `users` collection, with a `role` field to tell them apart. Each user has a `name`, `email` (unique, required), `password` (hashed before storage), `profilePicture` (URL string, optional), and a `bio` text field where students describe their background. The schema stores the extracted `skills` as an array of strings populated by the AI in Task 2. The `role` field must accept exactly three values: `jobSeeker`, `recruiter`, and `admin`, with `jobSeeker` as the default. Include a `status` field for recruiter accounts with values `pending`, `approved`, and `rejected` — only relevant when role is recruiter, but it can live on the User document. Finally, add a `createdAt` timestamp.

2. JobPost Schema

A listing needs to carry enough detail that a student can decide whether to apply without clicking elsewhere. Required fields include: `title` (e.g., "Backend Intern"), `company` (string), `description` (longer text), `requirements` (array of strings), `location` (string), `type` (`full-time`, `part-time`, or `internship`), and `salary` (number, optional). The `category` field will be auto-assigned by the AI when a recruiter creates a post — store it as a string (e.g., "Frontend", "Backend", "AI/ML", "DevOps", "Data Engineering", "Other"). Track `totalSlots` (how many candidates can be hired), `status` (`open` or `closed`), a reference to the `createdBy` user (Recruiter's `ObjectId`), and a `createdAt` timestamp.

3. Application Schema

Each application ties one student to one job and tracks where things stand in the hiring process. It must reference both the `user` (the applicant's `ObjectId`) and the `job` (the `JobPost`'s `ObjectId`) — both are required. Include a `coverLetter` text field (optional), a `status` field with values `pending`, `shortlisted`, and `rejected` (defaulting to `pending`), and an `appliedAt` timestamp.

Note: A student should not be able to apply to the same job twice. Consider how you would enforce that constraint at the schema or controller level — this is a design decision worth discussing in your PR reviews, even if the enforcement itself is implemented in Task 2.

7 Submission Requirements

- A GitHub repository with a branch for each team member, merged via Pull Requests into `main`.
- All three Mongoose schemas committed to the repository.
- Each team member must have at least one visible commit in the repository history.
- A `README.md` and a `.gitignore` must be present.

8 Task Deadline

Task 1 deadline is **Monday, 20th April 2026, 11:59 pm**

9 Submission

- a) Submit your GitHub repository link through the following form before the deadline:
<https://forms.gle/cm52Po9AUq13VUAF6>
- b) Invite the following GitHub user to your private repository so we can access your work:
Username: SEspring26